

Intro Desarrollo de Videojuegos 2026 - UNQ v.II

2 | Vectores - Físicas



Vectores



Los vectores se utilizan para representar puntos en el espacio, y tienen sobrada importancia sobretodo en física, para representar magnitudes como la **velocidad**, **fuerza**, etc.

• 01

En videojuegos también tenemos infinitos ejemplos en los que se usan estas magnitudes, como puede ser la velocidad de un coche en un juego de carreras o la fuerza que recibe un objeto después de que el jugador choque contra él, etc.

• 02

El uso de vectores en videojuegos proviene del mundo gráfico, donde se los utiliza para representar puntos en la pantalla o para computar la geometría de un objeto 3D y proyectarla.

• 03

Eje de coordenadas

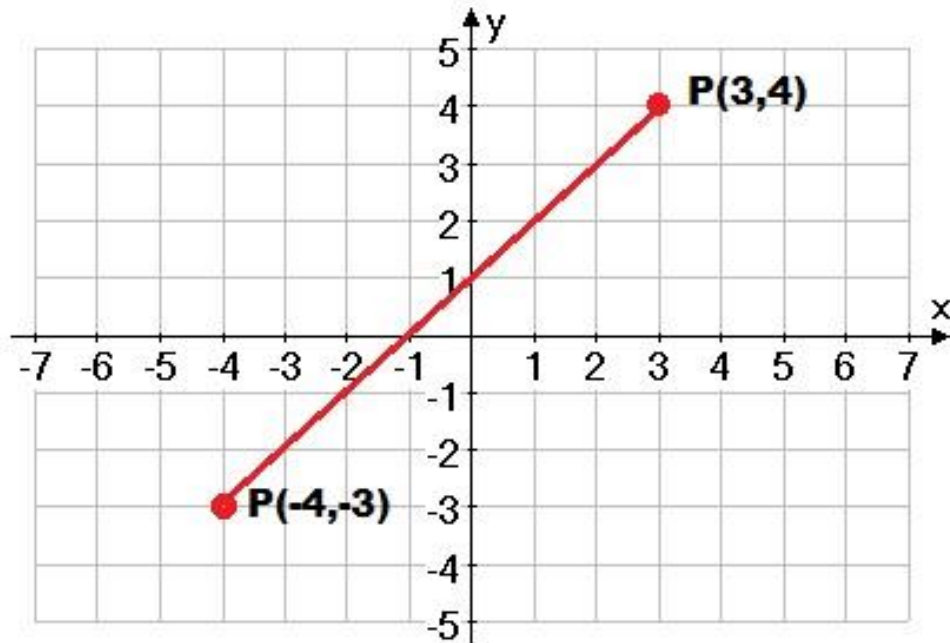


1

Plano cartesiano

Los ejes de coordenadas cartesianas son una parte fundamental de las matemáticas y la física, ya que nos sirven para representar funciones y también posiciones. En ellos, cada eje representa una dimensión, por lo que si hablamos de videojuegos encontraremos dos ejes básicos.

Representado por dos ejes X e Y que indican las dimensiones ancho y alto respectivamente. Se utiliza en juegos en dos dimensiones.

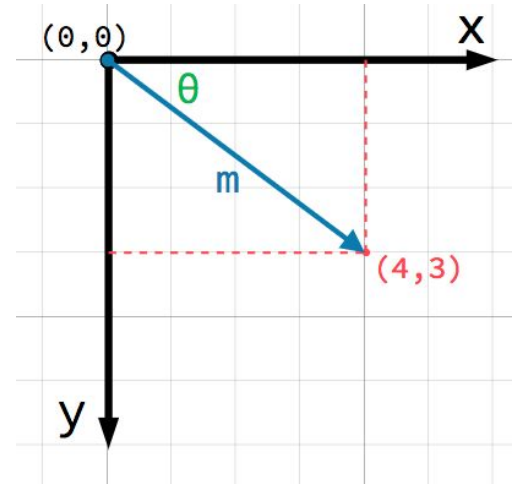
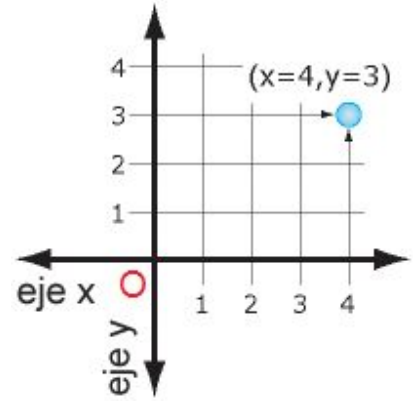
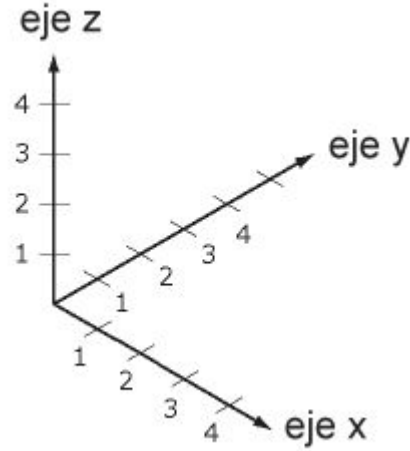




El origen

2

Cualquier posición en el plano 2D puede ser identificada por un par de números de esta manera. Sin embargo, también podemos pensar en la posición $(4, 3)$ como una **compensación** del punto $(0, 0)$, u **origen**. Dibuja una flecha apuntando desde el origen hasta el punto



¡Template con historia!

El proyecto que les compartimos fue creado **con mucho amor** hace unos años, en **Godot 3**.

Hoy Godot ya va por la versión **4.x**, y aunque **los desafíos se pueden encarar directamente en Godot 4**, es importante tener en cuenta algunos cambios en la sintaxis y estructura.

👁️ Les recomendamos revisar esta guía con tips clave para la migración:

 [Migrating to Godot 4.x](#)

💡 También pueden trabajar directamente en la última estable de 3 si lo prefieren, pero **el proceso de migrar puede ser un gran ejercicio** para entender mejor cómo evoluciona el motor.

¡A jugar, probar y experimentar! 🚀



Desafío #2

- Generar un proyecto en GODOT con:
 - Al Cuadrado negro (player) se le incorpora un nuevo rectángulo amarillo (mano) que apunta al mouse y tira bolitas.
 - Generar un rectángulo naranja como “Torreta” que puede ver al personaje y le dispara.

[Template inicial desafío 2](#)

[Demo objetivo final](#)



Video tutorial Desafío #2



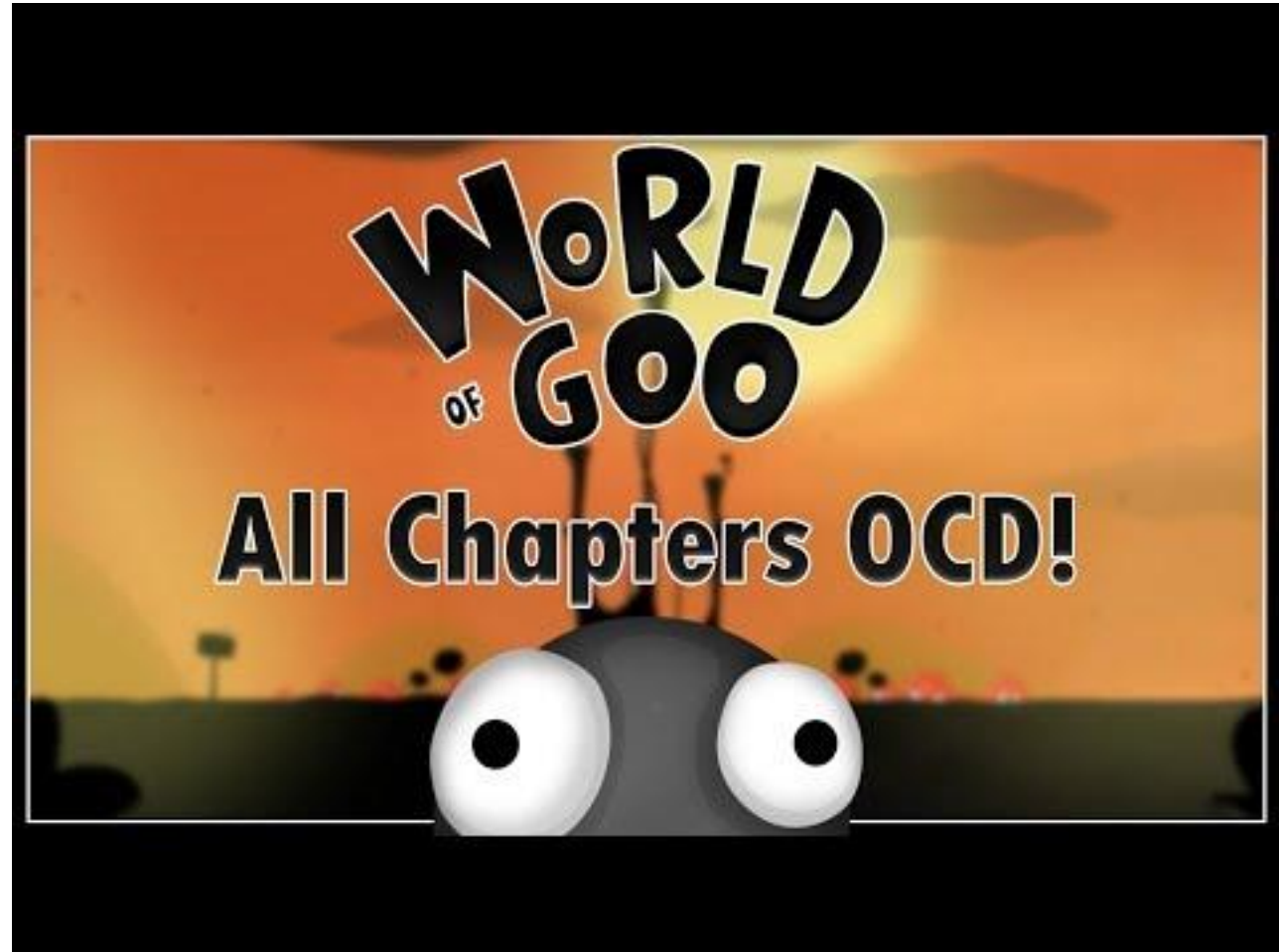
Hablemos de física

Y eso que mueve al mundo...



<https://www.osmos-game.com/>

Ejemplo



Ejemplo

https://store.steampowered.com/app/513450/Drop_Alive/



Otro

<https://killabunnies.com.ar/>



Drunk Race



PLAY

Published
1/31/21

Updated
1/31/21

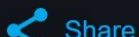
Max Players
16

Total Played
4893

If you don't like character control in Core Games, then you just haven't tried this game.

"Lost and Found" Global Game Jam entry.

SocialHub



Share



116



34

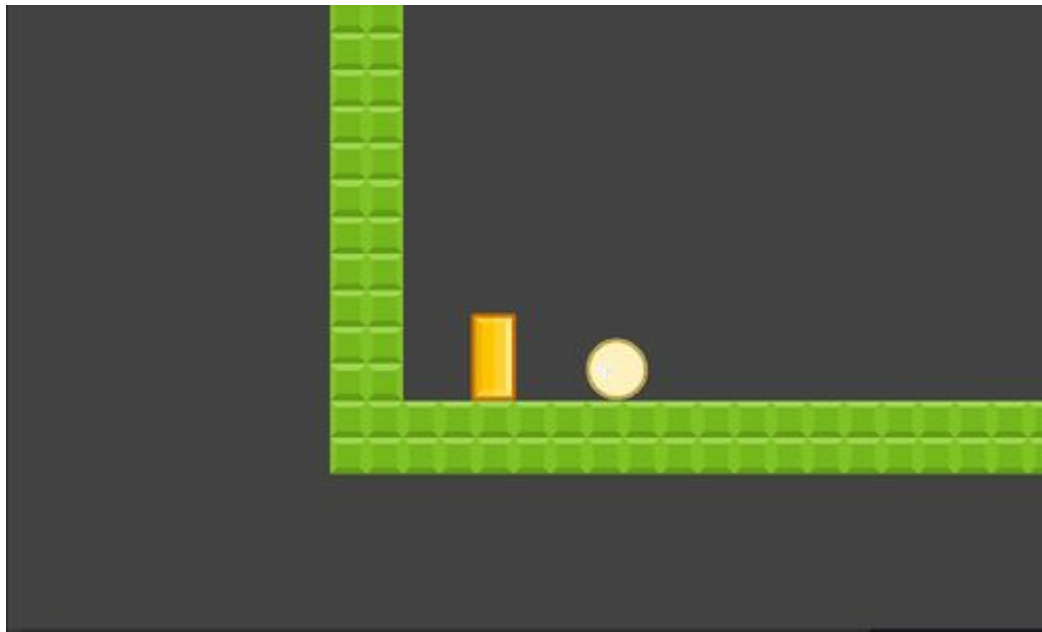
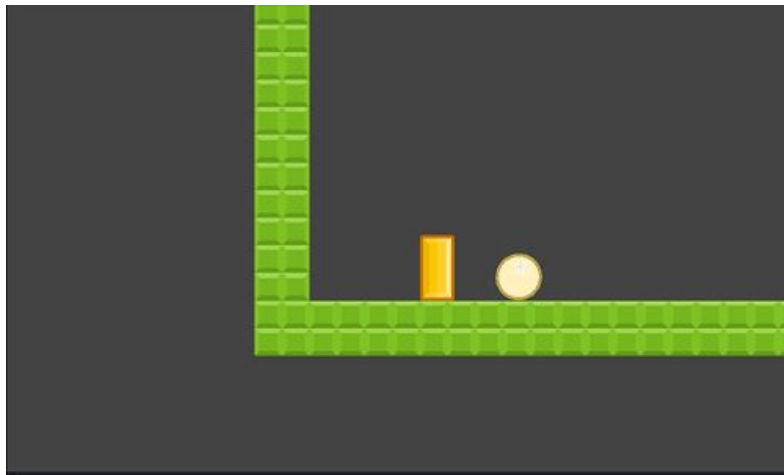
GAMEPLAY



ART OF
GRAVITY

[https://store.steampowered.com/app/642560/Art Of Gravity/](https://store.steampowered.com/app/642560/Art_of_Gravity/)

Kinematic and Rigid body



Documentación

[Introducción a la física](#)

[Areas](#)

[Static bodies \(Cuerpos estáticos\)](#)

[Rigid bodies \(Cuerpos rígidos\)](#)

[Kinematic bodies \(Cuerpos kinemáticos/cinemáticos\)](#)

[Raycasting](#)

Desafío #3

Generar un proyecto en GODOT con:

- Personaje que puede saltar y caer a un piso.
- Hay objetos físicos.
- La cámara sigue al player.
- Agrandar el mapa (más largo)
- Las torretas deben detectar al Player sin usar inyección de dependencias

Bonus:

- Torretas no pueden disparar si existe una pared en el medio.
- Los proyectiles pueden impactar a Player si es proyectil de Torreta y a Torreta si es proyectil de Player, y, al impactar, se eliminan.
- TurretSpawner sea una posición del mundo.

MATERIAL

Godot 4

- [Resolución del ejercicio anterior](#)
- [Plantilla inicial desafío 3](#)
- [Deploy final](#)

Godot 3

- [Resolución del ejercicio anterior](#)
- [Plantilla inicial desafío 3](#)
- [Deploy final](#)

La entrega debe contar con:

- Código fuente
- Export HTML con deploy jugable



Consideraciones de Godot 4

Godot 4 tiene cambios menores pero significativos respecto a Godot 3 en el manejo de las APIs de físicas. Al ver el video tutorial (hecho para Godot 3) hay que aplicar los siguientes cambios en los siguientes momentos para que funcione en Godot 4:

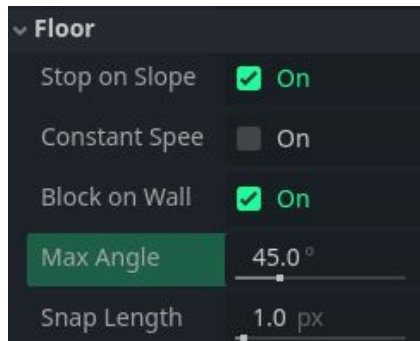
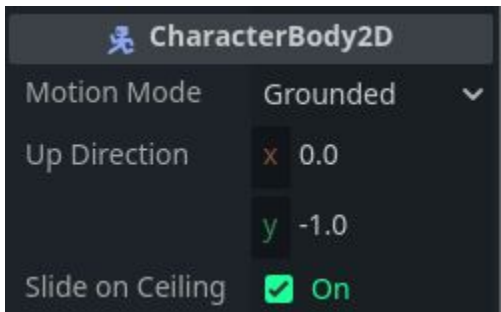
1 - KinematicBody2D ahora es CharacterBody2D

2 - Los parámetros de `move_and_slide()` pasaron a ser propiedades del CharacterBody2D editables por código o desde el inspector.

Por ejemplo, los parámetros para la línea:

```
move_and_slide_with_snap(velocity, snap_vector, FLOOR_NORMAL, true, 4, SLOPE_THRESHOLD)
```

Ahora son variables configurables desde el inspector, es decir, no se tienen que aplicar con cada llamado de `move_and_slide()`



Por lo que

- ajustando esas variables en el inspector
- en código aplicar los cambios a `velocity` (que ahora es built-in también)

- y llamando a `move_and_slide()` al final

Se logra lo mismo que en Godot 3 (Y es más sencillo)

3 - ¡Ya no hay inercia infinita default para los RigidBody2D! Ahora el comportamiento por defecto es que los CharacterBody2D no pueden empujar a los RigidBody2D.

Podemos configurar los nodos para tener 3 comportamientos diferentes:

A - Default, son inamovibles CharacterBody2D <-> RigidBody2D.

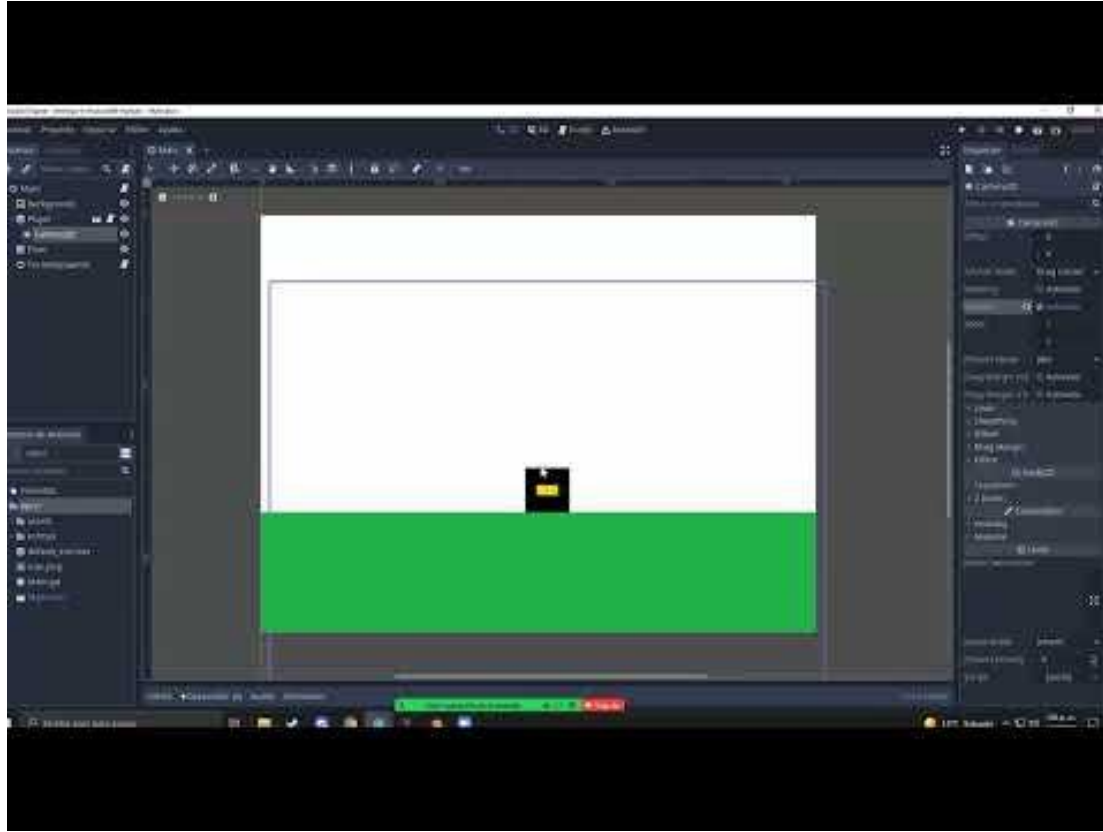
B - Comportamiento en Godot 3, CharacterBody2D tiene inercia infinita contra RigidBody2D.

C - Comportamiento complejo donde el CharacterBody2D trata a los RigidBody2D como pares y pueden trasladar impulsos entre sí sin inercia infinita con código.

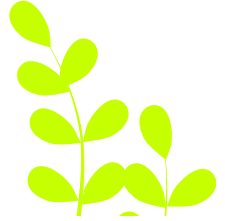
Video recomendado donde se explica este comportamiento en detalle



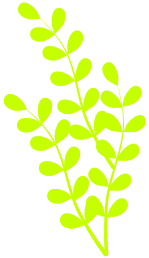
Tutorial Desafío #3



Tutorial



1. Formato de entrega y versionado



¿Cómo inicio el proyecto?

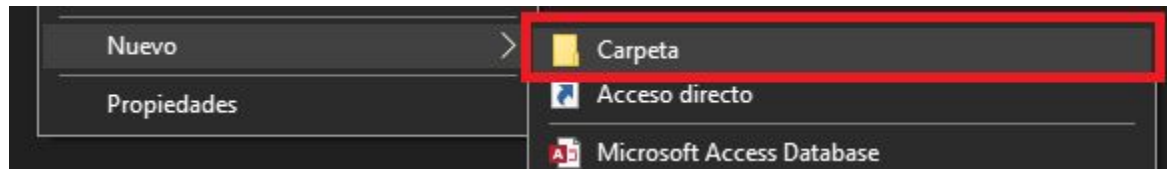
El formato de entrega es:

- El repositorio debe tener nombre “entregas-ipv-apellido-nombre”
- Las entregas deben realizarse en directorios dentro de ese repositorio, siguiendo el formato de nombre de “entrega-x-apellido-nombre”
- El nombre del proyecto de Godot (que se setea en la metadata del proyecto) también debe seguir el formato de las entregas, “entrega-x-apellido-nombre”

¿Cómo lo creamos y versionamos?

En el primer día, vino el repositorio...

El primer paso es crear un directorio raíz que nos servirá como repositorio *local*



Y la renombramos



Luego inicializamos el directorio como un repositorio git (asegurándonos primero de tener [instalado Git](#))

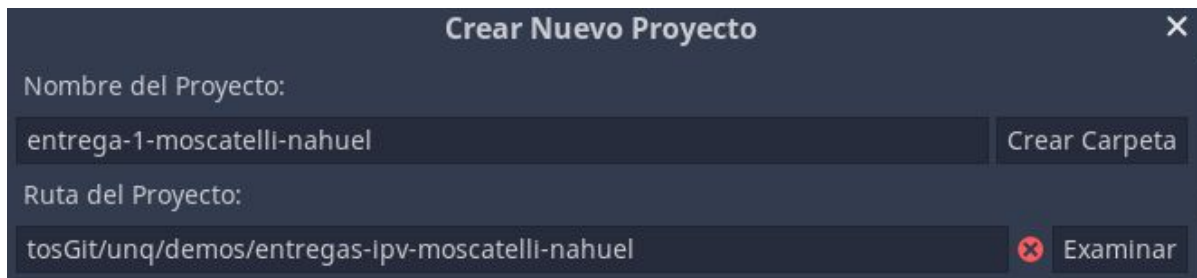
Desde la consola (como Git Bash):

```
R4nKF1v3@DESKTOP-G1DQF91 MINGW64 ~/ProyectosGit/unq/demos/entregas-ipv-moscattelli-nahuel
$ git init
Initialized empty Git repository in D:/ProyectosGit/unq/demos/entregas-ipv-moscattelli-nahuel/.git/
R4nKF1v3@DESKTOP-G1DQF91 MINGW64 ~/ProyectosGit/unq/demos/entregas-ipv-moscattelli-nahuel (main)
$ |
```

¡Ahora, quiero mi proyecto!

Ahora queremos crear el proyecto de Godot. Esto podemos hacerlo directamente desde Godot.

En el launcher de Godot, le damos a 'Nuevo Proyecto' y colocamos el nombre del proyecto acorde al formato



Crear Nuevo Proyecto

Nombre del Proyecto:

entrega-1-moscatelli-nahuel

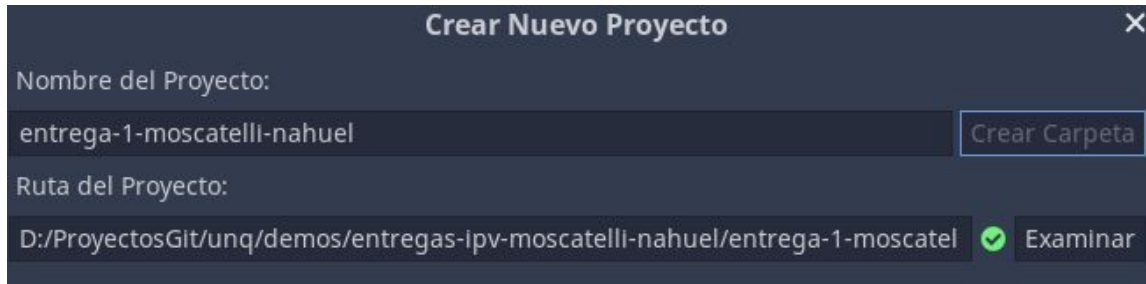
Crear Carpeta

Ruta del Proyecto:

tosGit/unq/demos/entregas-lpv-moscatelli-nahuel

Examinar

Y, asegurandonos que en 'Ruta del Proyecto' esta la ruta del repositorio, le hacemos click a 'Crear Carpeta', lo que nos creará un nuevo directorio en esa ruta con el nombre del proyecto



Crear Nuevo Proyecto

Nombre del Proyecto:

entrega-1-moscatelli-nahuel

Crear Carpeta

Ruta del Proyecto:

D:/ProyectosGit/unq/demos/entregas-lpv-moscatelli-nahuel/entrega-1-moscatel

Examinar

Hecho esto, se puede comenzar a trabajar!

¿Y cómo lo versiono?

Desde la consola chequeamos el status ('git status') en el repositorio local, allí debe de aparecer como 'untracked' el repositorio que corresponde al proyecto de entrega

```
R4nKF1v3@DESKTOP-G1DQF91 MINGW64 ~/ProyectosGit/unq/demos/entregas-ipv-moscatelli-nahuel (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    entrega-1-moscatelli-nahuel/

nothing added to commit but untracked files present (use "git add" to track)
```

Lo agregamos con 'git add <file>' para situarlo en staged

```
R4nKF1v3@DESKTOP-G1DQF91 MINGW64 ~/ProyectosGit/unq/demos/entregas-ipv-moscatelli-nahuel (main)
$ git add entrega-1-moscatelli-nahuel/
```

Y luego hacemos 'git commit -m "mensaje del commit"' para confirmar la transacción

```
R4nKF1v3@DESKTOP-G1DQF91 MINGW64 ~/ProyectosGit/unq/demos/entregas-ipv-moscatelli-nahuel (main)
$ git commit -m "Creada entrega 1"
```


¡Pero yo quiero compartirlo!

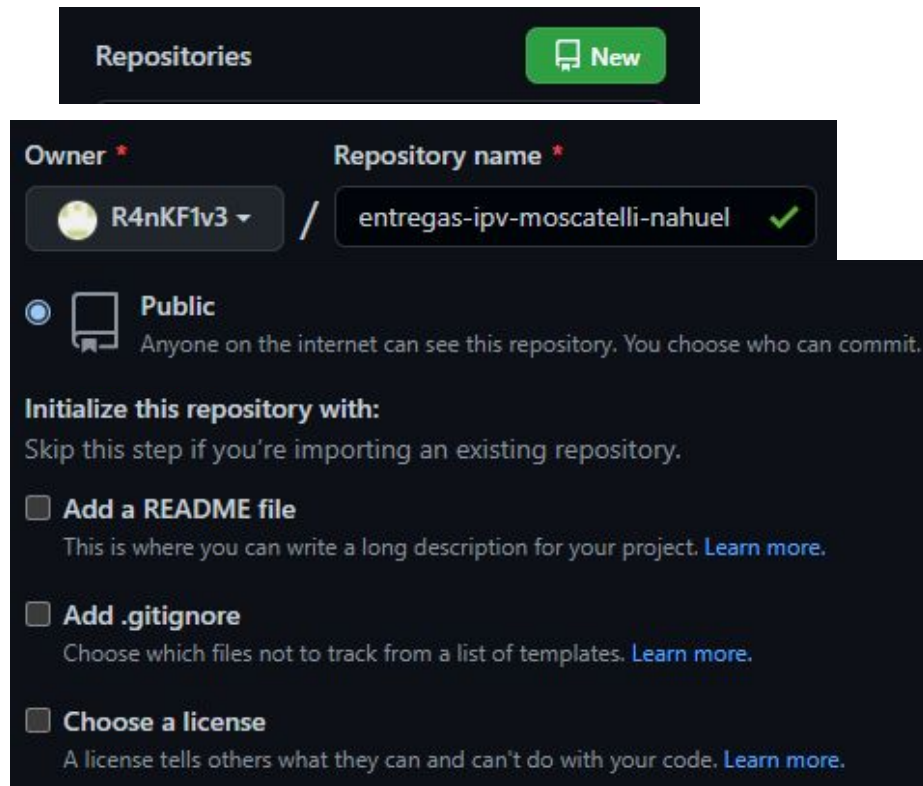
Para eso necesitamos crear un Repositorio Remoto. En la cátedra aconsejamos usar [Github](#), pero alternativas como [Gitlab](#) o similares también son válidas.

Primero buscamos el boton de crear nuevo repositorio en Github (teniendo una cuenta ya creada)

En la pestaña nueva, le ponemos el nombre al repositorio con el formato de entrega

Nos aseguramos que la visibilidad este en “Público”

Y también nos aseguramos que no este tildada ninguna de las opciones referidas a la creación de un README, .gitignore o licencias



The screenshot shows the 'New Repository' form on GitHub. At the top, there's a dark header with the word 'Repositories' and a green 'New' button with a repository icon. Below this, the form is divided into sections. The first section has two input fields: 'Owner' with a dropdown menu showing 'R4nKF1v3' and a 'Repository name' field containing 'entregas-ipv-moscatelli-nahuel' with a green checkmark. Below these is a radio button for 'Public' (selected) and a description: 'Anyone on the internet can see this repository. You choose who can commit.' The next section is titled 'Initialize this repository with:' and includes a note: 'Skip this step if you're importing an existing repository.' There are three checkboxes: 'Add a README file' (unchecked), 'Add .gitignore' (unchecked), and 'Choose a license' (unchecked). Each checkbox has a brief description and a 'Learn more' link.

Repositories New

Owner * Repository name *

R4nKF1v3 / entregas-ipv-moscatelli-nahuel ✓

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

☐ **Add .gitignore**
Choose which files not to track from a list of templates. [Learn more.](#)

☐ **Choose a license**
A license tells others what they can and can't do with your code. [Learn more.](#)

GITHUB

¿Qué es GitHub?

GitHub es una plataforma de desarrollo colaborativo de software para alojar proyectos utilizando el sistema de control de versiones [Git](#).

¿Para qué sirve?

GitHub aloja tu repositorio de código y te brinda herramientas muy útiles para el trabajo en equipo, dentro de un proyecto.

Además de eso, puedes contribuir a mejorar el software de los demás. Para poder alcanzar esta meta, GitHub provee de funcionalidades para hacer un fork y solicitar pulls.

Realizar un fork es simplemente clonar un repositorio ajeno (genera una copia en tu cuenta), para eliminar algún bug o modificar cosas de él. Una vez realizadas tus modificaciones puedes enviar un pull al dueño del proyecto. Éste podrá analizar los cambios que has realizado fácilmente, y si considera interesante tu contribución, adjuntarlo con el repositorio original.

¡A subir el código!

Ahora tenes un repositorio remoto limpio, pero necesitas vincular tu repositorio local para poder subir tu código

Si seguiste el tutorial hasta ahora, te va a salir una sugerencia automática de github con los comandos a introducir para subir rápido tu repositorio ya existente.

...or push an existing repository from the command line

```
git remote add origin git@github.com:R4nKF1v3/entregas-ipv-moscatelli-nahuel.git
git branch -M main
git push -u origin main
```

Estos comandos funcionan independientemente de la plataforma

¡Y listo! Ahora, cuando quieras subir tu código, solo tenes que repetir el comando 'git push' (sin los demás parámetros) para que lo suba directamente a la branch seleccionada



Gracias!
Nos vemos en la próxima